

COMPUTER ARITHMETIC/ ARITHMETIC ALGORITHM/I

1. Addition and Subtraction With Signed –Magnitude Data:

We designate the magnitude of the two numbers by A and B. Where the signed numbers are added or subtracted, we find that there are eight different conditions to consider, depending on the sign of the numbers and the operation performed.

When the signs of A and B are same, add the two magnitudes and attach the sign of result is that of A. When the signs of A and B are not same, compare the magnitudes and subtract the smaller number from the larger. Choose the sign of the result to be the same as A, if $A > B$ or the complement of the sign of A if $A < B$. If the two magnitudes are equal, subtract B from A and make the sign of the result will be positive.

Eight different conditions:

Operation	Add Magnitudes	SUBTRACT Magnitudes		
		A > B	A < B	A = B
(+A) + (+B)	+ (A + B)			
(+A) + (-B)		+ (A - B)	- (B - A)	+ (A - B)
(-A) + (+B)		- (A - B)	+ (B - A)	+ (A - B)
(-A) + (-B)	- (A + B)			
(+A) - (+B)		+ (A - B)	- (B - A)	+ (A - B)
(+A) - (-B)	+ (A + B)			
(-A) - (+B)	- (A + B)			
(-A) - (-B)		- (A - B)	+ (B - A)	+ (A - B)

Table 10.1 Addition and Subtraction of Signed-Magnitude Numbers

Hardware implementation: To implement the two arithmetic operations with hardware, we have to store numbers into two registers A and B. Let A_s and B_s be two flip-flops that hold the corresponding signs. The results are transferred to A and A_s . A and A_s together form an accumulator.

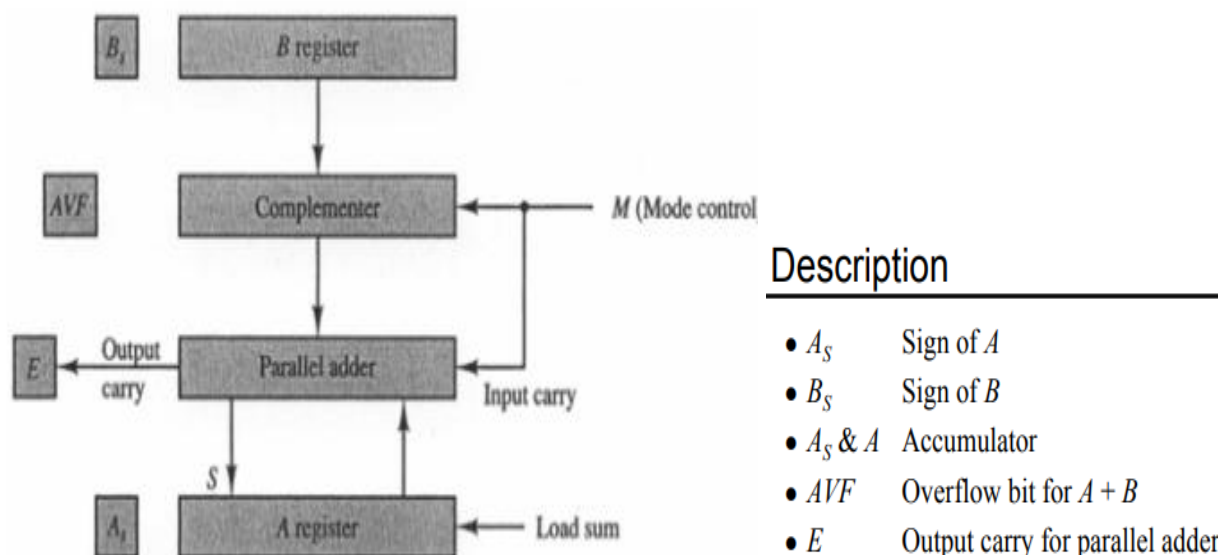


Figure: Hardware Architecture for Addition and Subtraction of Signed-Magnitude Numbers

Explanation for Hardware block diagram and Flowchart for Addition and Subtraction of Signed –Magnitude Numbers.

We need:

- Two registers A and B and sign flip-flops As and Bs
- A magnitude comparator: to check if $A > B$, $A < B$, or $A = B$
- A parallel adder: to perform $A + B$
- Two parallel subtractions: for $A - B$ and $B - A$
- The sign relationships are determined from an X-OR gate with As and Bs as inputs.

Block Diagram Description

- Hardware above consists of registers A and B and sign flip-flops As and Bs.
- The complementary provides an output of B or B' depending on mode input M.
- When $M=0$, the output of B is transferred to the adder, the input carry is 0 and thus output of adder is $A+B$. Output carry is transferred to flip-flop E, where it can be checked to determine overflow. When $E=1$, it is overflow, otherwise, not. The $E=1$ is transferred to the AVF (add-overflow-flip-flop) which holds the overflow bit when A and B are added.
- When $M=1$, 1's complement of B is applied to the adder, input carry is 1 and output is $S=A+B'+1$ (i.e. $A-B$). Output carry is transferred to flip-flop E, where it can be checked to determine the relative magnitude of two numbers.

Hardware Algorithm

- The flowchart for the hardware algorithm is given below:
- As and Bs are compared by an X-OR gate. If output=0, signs are identical, if 1, signs are different.
- For add operation, identical signs dictate addition of magnitudes. For subtraction, different signs dictate magnitudes be added.
- Magnitudes are with a micro operation $EA = A+B$ (EA is register that combines E and A). If $E=1$, overflow occurs and is transferred to AVF.
- Two magnitudes are subtracted if signs are different for add operation and identical for subtract operation.
- Magnitudes are subtracted with a micro operation $EA = A+B'+1$.
- No overflow occurs if the numbers are subtracted, so AVF is cleared to 0.
- If $E=1$, it indicates $A \geq B$ and the result in A is correct. If the number in A is 0, the sign As must be made positive ($As=0$) to avoid negative zero.
- If $E=0$, it indicates that $A < B$ for which it is necessary to take 2's complement of value in A. the micro operation is $A = A'+1$. The sign of As must be sign of Bs (i.e. $As = As'$).

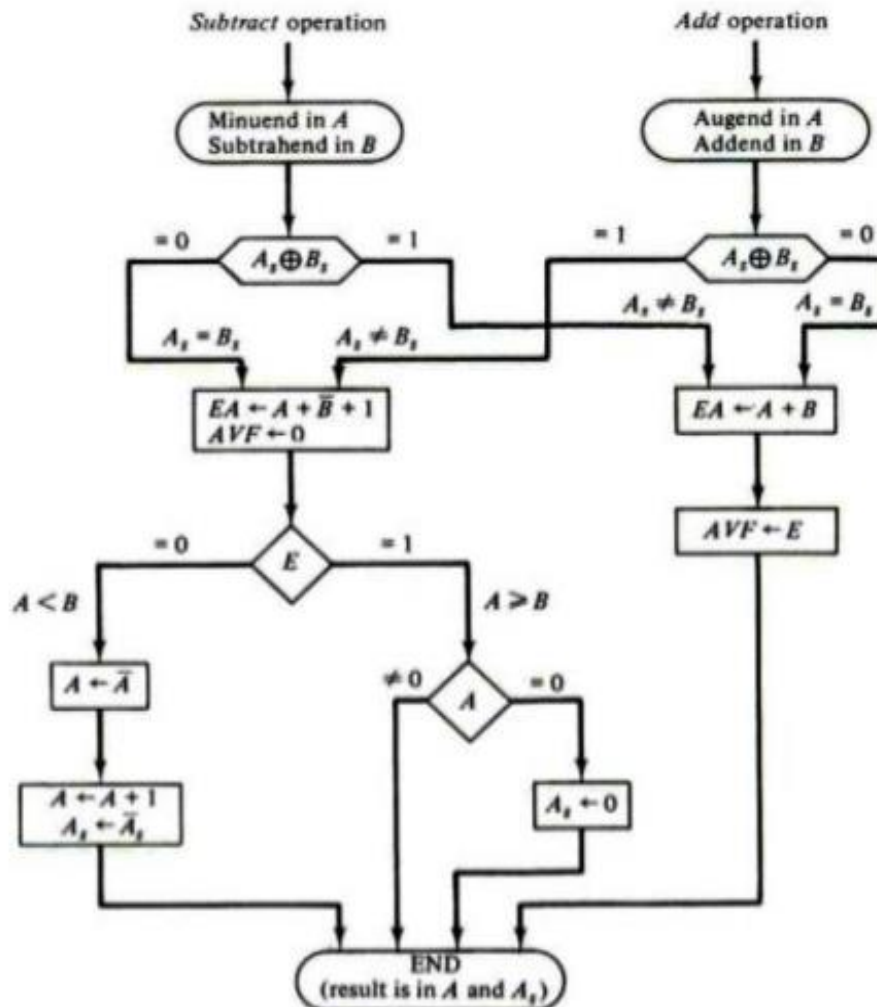


Figure: Flowchart

Addition and Subtraction with Signed -2's Complement Numbers:

- In signed 2's complement representation, the leftmost bit represents sign (0 for positive and 1 for negative). If sign bit is 1, entire number is represented in 2's complement form.
- **Addition:** Sign bit is treated as other bits of the number. Carry out of the sign bit is discarded.
- **Subtraction:** It consists of first taking 2's complement of subtrahend and then adding it to minuend. When two numbers of n -digit each are added, the sum occupies $n+1$ bits. Overflow occurs which is detected by applying last two carries out of the addition to XOR gate. The overflow occurs only if output of the gate is 1.

Hardware Implementation

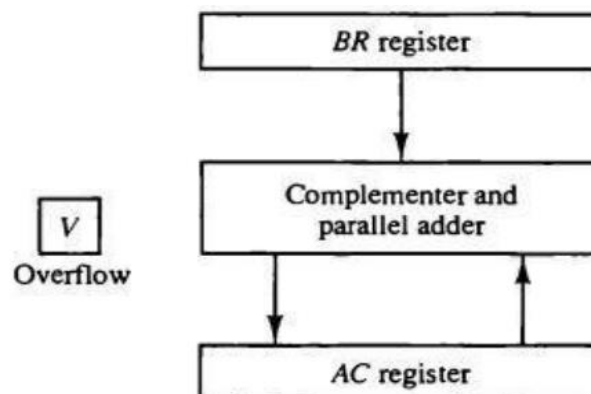


Fig: Hardware for signed-2's complement data addition and subtraction

- Register configuration is same as signed-magnitude representation except sign bits are not separated. The leftmost bits in AC and BR represent sign bits.
- Significant Difference: Sign bits are added together with the other bits in complementer and parallel adder. The overflow flip-flop V is set to 1 if there is an overflow. Output carry in this case is discarded.

Hardware Algorithm

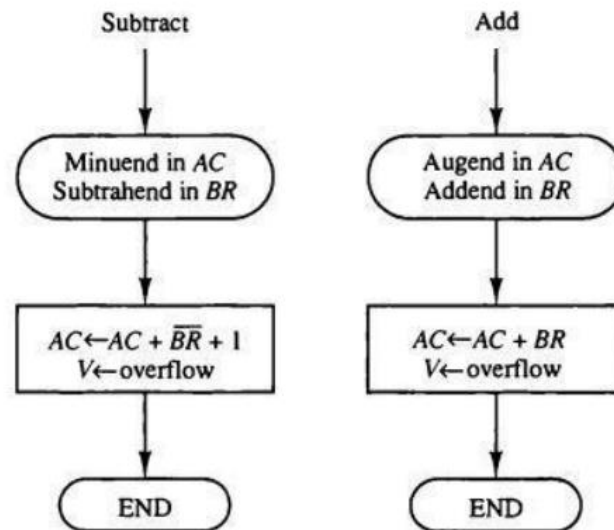


Fig: Algorithm for signed-2's complement data addition and subtraction

- Comparing this with its signed-magnitude counterpart, it is much easier to add and subtract numbers. For this reason, most computers adopt this representation over the more familiar signed magnitude.

Example: 33+ (-35)

AC = 33 = 00100001

BR = -35 = 2's complement of 35 = 11011101

AC+BR = 11111110 = -2

BCD Arithmetic

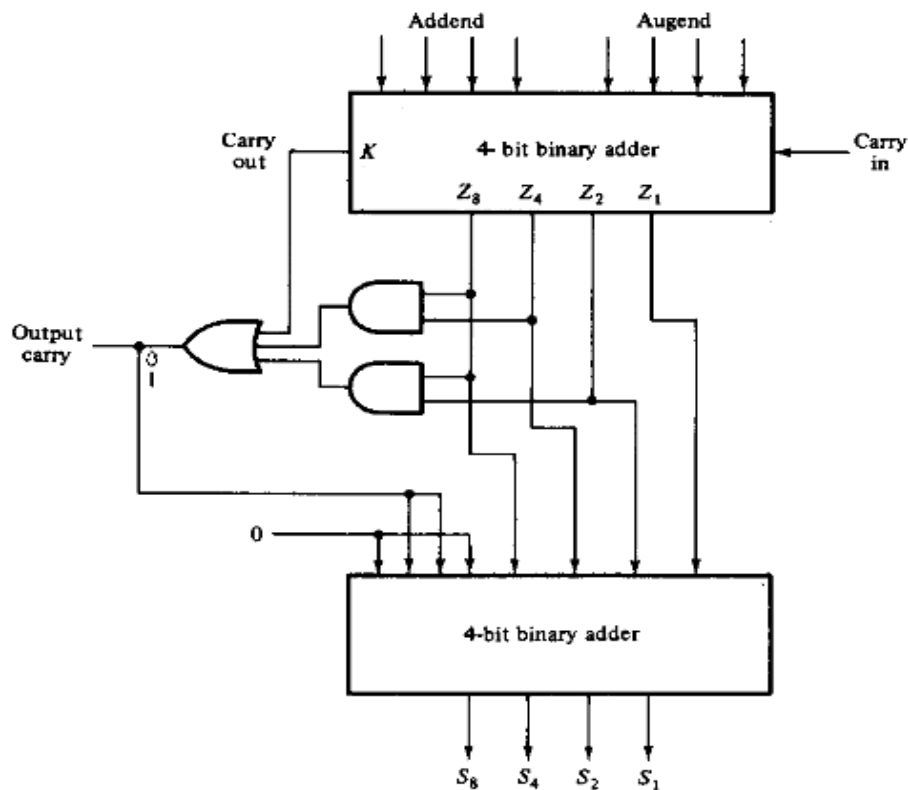
BCD Adder

- The next higher-order pair of digits, together with the input-carry, is then added to produce their binary sum. If this result is equal to or greater than 1010, then it is corrected by adding 0110.
- The procedure is repeated until all decimal digits are added.
- It is obvious that a correction is needed when the binary sum has an output carry $K = 1$. The other six combinations from 1010 to 1111 that need a correction have a 1 in position Z8.
- To differentiate them from binary 1000 and 1001, which also have a 1 in position Z8, we specify further that either Z4 or Z2 must have a 1. The condition for a correction and an output-carry can be expressed by the Boolean function

$$C = K + Z_8 Z_4 + Z_8 Z_2$$

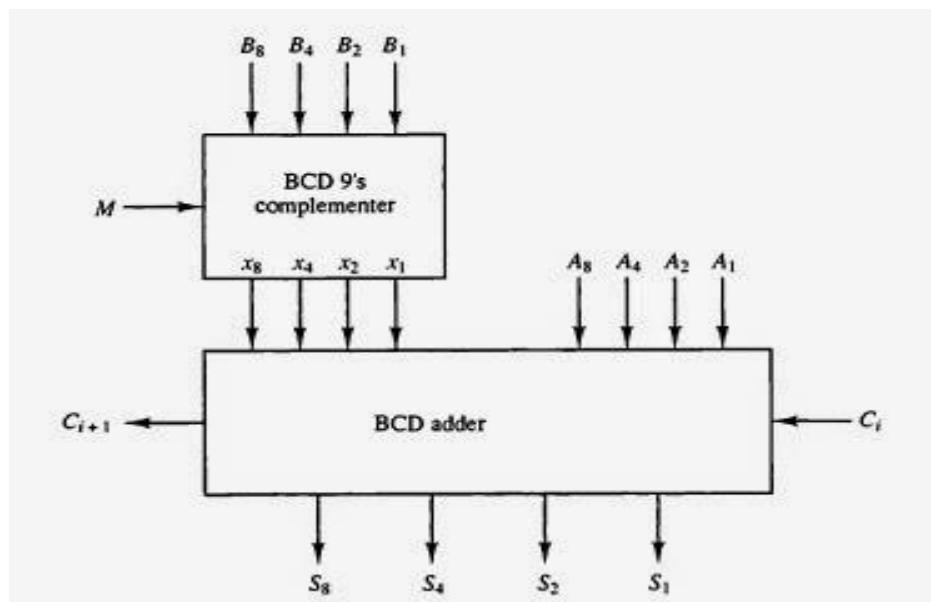
When $C = 1$, we need to add 0110 to the binary sum and provide an output-carry for the next stage.

- A BCD adder is circuit that adds two BCD digits in parallel and generates a sum digit also in BCD. A BCD adder must include the correction logic in its internal construction. To add 0110 to the binary sum, we use a second 4-bit binary adder.
- The two decimal digits, together with the input carry, are first added in the top 4-bit binary adder to produce the binary sum.
- When the output-carry is equal to 0, nothing is added to the binary sum through the bottom 4-bit binary adder. The output carry generated from the bottom binary adder may be ignored, since it supplies information already available in the output-carry terminal. It can be showed in below figure



BCD subtractor

Combining the above 9's complement generator and the BCD adder it is able to design the 1 -digit BCD adder subtractor as follows.



- Subtraction of two decimal numbers needs a subtractor circuit that is different from a BCD adder. We perform the subtraction by taking the 9's or 10's complement of the subtrahend and adding it to the minuend.
- The 9's complement is formed by using a circuit that subtracts each BCD digit from 9.
- First method, the 9's complement of BCD 0111(decimal 7) is computed by first complementing each bit to obtain 1000. Adding binary 1010 and discarding the carry, we obtain 0010 (decimal 2).
- By the second method, we add 0110 to 0111 to obtain 1101. Complementing each bit, we obtain the required result of 0010.

Examples

- | | |
|------------------|---------------------|
| 0111 decimal 7 | 0111 |
| 1000 complement | +0110 add decimal 6 |
| +1010 decimal 10 | 1101 |
| 1 0010 decimal 2 | 0010 complement |

- When this circuit is combined to a BCD adder, we get a BCD adder/subtractor. Let the subtrahend (or addend) digit be denoted by the four binary variables B8, B4, B2, and B1.
- Let M be a mode bit that controls the add/subtract operation. When M = 0, the two digits are added; when M = 1, the digits are subtracted. Let the binary variables x8, x4, x2, and x1 be the outputs of the 9's complement circuit.

By an examination of the truth table for the circuit, it may be observed that B1 should always be complemented; B2 is always the same in the 9's complement as in the original digit; x4 is 1 when the exclusive OR of B2 and B4 is 1; and x8 is 1 when B8B4B2 = 000. The Boolean functions for the 9's complement circuit are

$$x1 = B1 M' + B'1 M \quad x2 = B2$$

$$x4 = B4M' + (B'4B2 + B4B'2)M$$

$$x8 = B8M' + B'8B4'B'2M$$

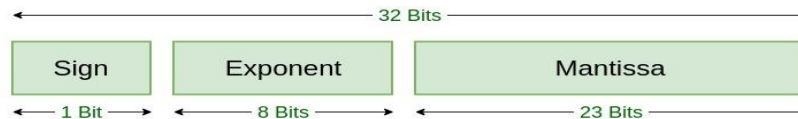
- From these equations we see that $x = B$ when $M = 0$. When $M = 1$, the x equals to the 9's complement of B .
- One stage of a decimal arithmetic unit that can be used to add or subtract two BCD digits is given in Fig. It has of a BCD adder and a 9's complementer.
- The mode M controls the operation of the unit. With $M = 0$, the S outputs form the sum of A and B . With $M = 1$, the S outputs form the sum of A plus the 9's complement of B .

In 10's complement method, to subtract the two decimal numbers let $M = 1$ and apply a 1 to the input carry $C1$ of the first stage. The outputs will form the sum of A plus the 10's complement of B , which is equivalent to a subtraction operation if the carry-out of the last stage is discarded.

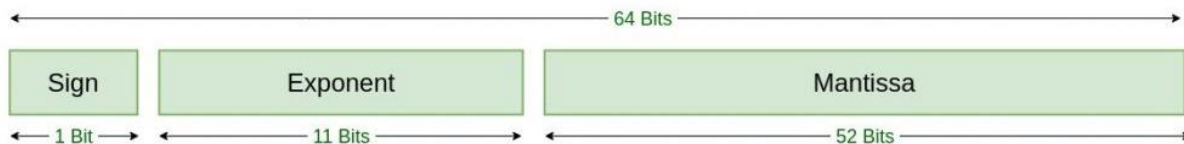
Floating-point Arithmetic operations

Floating point representation of number

Digital Computers use Binary number system to represent all types of information inside the computers. Digital representations are easier to design, storage is easy, accuracy and precision are greater. Most of the binary floating-point representations follow the IEEE-754 standard. The data type float uses IEEE 32-bit single precision format and the data type double uses IEEE 64-bit double precision format.



Single Precision
IEEE 754 Floating-Point Standard



Double Precision
IEEE 754 Floating-Point Standard

The IEEE 754 standard defines several different precisions.

- Single precision numbers include an 8-bit exponent field and a 23-bit fraction, for a total of 32 bits.
- Double precision numbers have an 11-bit exponent field and a 52-bit fraction, for a total of 64 bits.
- The sign bit is 0 for positive numbers and 1 for negative numbers. The exponent range for normalized numbers is $[-126, 127]$ for single precision, $[-1022, 1023]$ for double.

Example: 85.125

$$85 = 1010101$$

$$0.125 = 001$$

$$85.125 = 1010101.001$$

$$= 1.010101001 \times 2^6$$

$$\text{sign} = 0$$

1. Single precision:

$$\text{biased exponent } 127 + 6 = 133$$

$$133 = 10000101$$

$$\text{Normalised mantissa} = 010101001$$

we will add 0's to complete the 23 bits. The IEEE 754 Single precision is:

$$= 0 \ 10000101 \ 01010100100000000000000$$

2. Double precision:

biased exponent $1023+6=1029$

$$1029 = 10000000101$$

Normalised mantissa = 010101001

we will add 0's to complete the 52 bits

The IEEE 754 Double precision is:

$$= 0 \text{ } 10000000101 \text{ } 010\overline{1}01001000000000000000000000000000000000000$$

10-5 Floating-Point Arithmetic Operations

Many high-level programming languages have a facility for specifying floating-point numbers. The most common way is to specify them by a *real* declaration statement as opposed to fixed-point numbers, which are specified by an *integer* declaration statement. Any computer that has a compiler for such high-level programming language must have a provision for handling floating-point arithmetic operations. The operations are quite often included in the internal hardware. If no hardware is available for the operations, the compiler must be designed with a package of floating-point software subroutines. Although the hardware method is more expensive, it is so much more efficient than the software method that floating-point hardware is included in most computers and is omitted only in very small ones.

Basic Considerations

Floating-point representation of data was introduced in Sec. 3-4. A floating-point number in computer registers consists of two parts: a mantissa m and an exponent e . The two parts represent a number obtained from multiplying m times a radix r raised to the value of e ; thus

$$m \times r^e$$

The mantissa may be a fraction or an integer. The location of the radix point and the value of the radix r are assumed and are not included in the registers. For example, assume a fraction representation and a radix 10. The decimal number 537.25 is represented in a register with $m = 53725$ and $e = 3$ and is interpreted to represent the floating-point number

$$.53725 \times 10^3$$

A floating-point number is normalized if the most significant digit of the mantissa is nonzero. In this way the mantissa contains the maximum possible number of significant digits. A zero cannot be normalized because it does not have a nonzero digit. It is represented in floating-point by all 0's in the mantissa and exponent.

Floating-point representation increases the range of numbers that can be accommodated in a given register. Consider a computer with 48-bit words. Since one bit must be reserved for the sign, the range of fixed-point integer numbers will be $\pm(2^{47} - 1)$, which is approximately $\pm 10^{14}$. The 48 bits can be used to represent a floating-point number with 36 bits for the mantissa and 12 bits for the exponent. Assuming fraction representation for the mantissa and taking the two sign bits into consideration, the range of numbers that can be accommodated is

$$\pm(1 - 2^{-35}) \times 2^{2047}$$

This number is derived from a fraction that contains 35 1's, an exponent of 11 bits (excluding its sign), and the fact that $2^{11} - 1 = 2047$. The largest number that can be accommodated is approximately 10^{615} , which is a very large number. The mantissa can accommodate 35 bits (excluding the sign) and if considered as an integer it can store a number as large as $(2^{35} - 1)$. This is approximately equal to 10^{10} , which is equivalent to a decimal number of 10 digits.

Computers with shorter word lengths use two or more words to represent a floating-point number. An 8-bit microcomputer may use four words to represent one floating-point number. One word of 8 bits is reserved for the exponent and the 24 bits of the other three words are used for the mantissa.

Arithmetic operations with floating-point numbers are more complicated than with fixed-point numbers and their execution takes longer and requires more complex hardware. Adding or subtracting two numbers requires first an alignment of the radix point since the exponent parts must be made equal before adding or subtracting the mantissas. The alignment is done by shifting one mantissa while its exponent is adjusted until it is equal to the other exponent. Consider the sum of the following floating-point numbers:

$$.5372400 \times 10^2$$

$$+ .1580000 \times 10^{-1}$$

It is necessary that the two exponents be equal before the mantissas can be added. We can either shift the first number three positions to the left, or shift the second number three positions to the right. When the mantissas are stored in registers, shifting to the left causes a loss of most significant digits. Shifting to the right causes a loss of least significant digits. The second method is preferable because it only reduces the accuracy, while the first method may cause an error. The usual alignment procedure is to shift the mantissa that has

the smaller exponent to the right by a number of places equal to the difference between the exponents. After this is done, the mantissas can be added:

$$\begin{array}{r} .5372400 \times 10^2 \\ + .0001580 \times 10^2 \\ \hline .5373980 \times 10^2 \end{array}$$

When two normalized mantissas are added, the sum may contain an overflow digit. An overflow can be corrected easily by shifting the sum once to the right and incrementing the exponent. When two numbers are subtracted, the result may contain most significant zeros as shown in the following example:

$$\begin{array}{r} .56780 \times 10^5 \\ - .56430 \times 10^5 \\ \hline .00350 \times 10^5 \end{array}$$

A floating-point number that has a 0 in the most significant position of the mantissa is said to have an *underflow*. To normalize a number that contains an underflow, it is necessary to shift the mantissa to the left and decrement the exponent until a nonzero digit appears in the first position. In the example above, it is necessary to shift left twice to obtain $.35000 \times 10^3$. In most computers, a normalization procedure is performed after each operation to ensure that all results are in a normalized form.

Floating-point multiplication and division do not require an alignment of the mantissas. The product can be formed by multiplying the two mantissas and adding the exponents. Division is accomplished by dividing the mantissas and subtracting the exponents.

The operations performed with the mantissas are the same as in fixed-point numbers, so the two can share the same registers and circuits. The operations performed with the exponents are compare and increment (for aligning the mantissas), add and subtract (for multiplication and division), and decrement (to normalize the result). The exponent may be represented in any one of the three representations: signed-magnitude, signed-2's complement, or signed-1's complement.

A fourth representation employed in many computers is known as a *biased* exponent. In this representation, the sign bit is removed from being a separate entity. The bias is a positive number that is added to each exponent as the floating-point number is formed, so that internally all exponents are positive. The following example may clarify this type of representation. Consider an exponent that ranges from -50 to 49 . Internally, it is represented by two digits (without a sign) by adding to it a bias of 50 . The exponent register contains the number $e + 50$, where e is the actual exponent. This way, the exponents are represented in registers as positive numbers in the range of 00

to 99. Positive exponents in registers have the range of numbers from 99 to 50. The subtraction of 50 gives the positive values from 49 to 0. Negative exponents are represented in registers in the range from 49 to 00. The subtraction of 50 gives the negative values in the range of -1 to -50.

The advantage of biased exponents is that they contain only positive numbers. It is then simpler to compare their relative magnitude without being concerned with their signs. As a consequence, a magnitude comparator can be used to compare their relative magnitude during the alignment of the mantissa. Another advantage is that the smallest possible biased exponent contains all zeros. The floating-point representation of zero is then a zero mantissa and the smallest possible exponent.

In the examples above, we used decimal numbers to demonstrate some of the concepts that must be understood when dealing with floating-point numbers. Obviously, the same concepts apply to binary numbers as well. The algorithms developed in this section are for binary numbers. Decimal computer arithmetic is discussed in the next section.

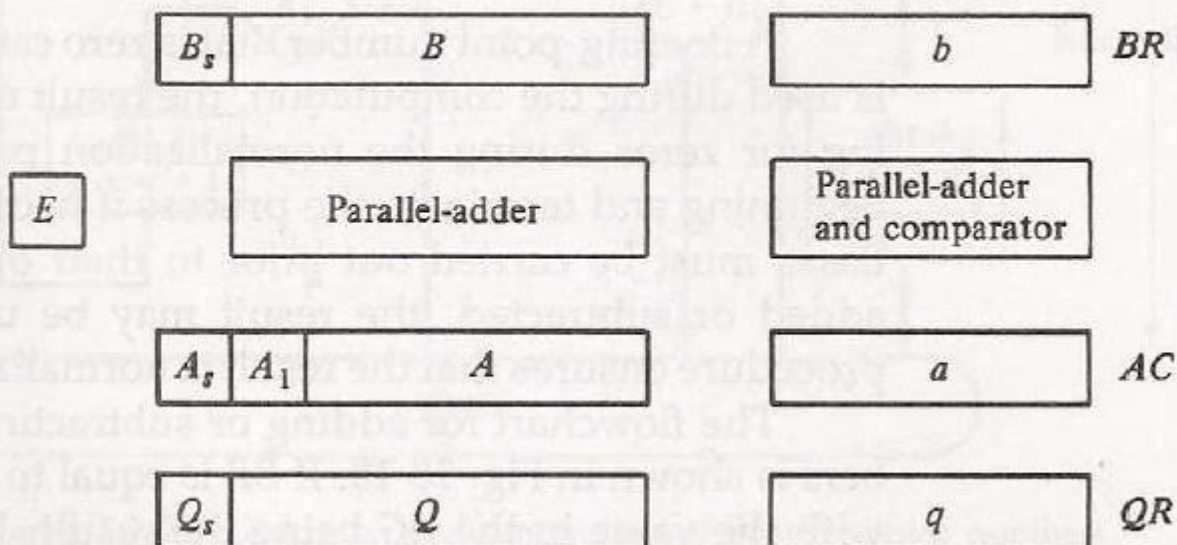
Register Configuration

The register configuration for floating-point operations is quite similar to the layout for fixed-point operations. As a general rule, the same registers and adder used for fixed-point arithmetic are used for processing the mantissas. The difference lies in the way the exponents are handled.

The register organization for floating-point operations is shown in Fig. 10-14. There are three registers, *BR*, *AC*, and *QR*. Each register is subdivided into two parts. The mantissa part has the same uppercase letter symbols as in fixed-point representation. The exponent part uses the corresponding lower-case letter symbol.

It is assumed that each floating-point number has a mantissa in signed-magnitude representation and a biased exponent. Thus the *AC* has a mantissa

Figure 10-14 Registers for floating-point arithmetic operations.



whose sign is in A_s and a magnitude that is in A . The exponent is in the part of the register denoted by the lowercase letter symbol a . The diagram shows explicitly the most significant bit of A , labeled by A_1 . The bit in this position must be a 1 for the number to be normalized. Note that the symbol AC represents the entire register, that is, the concatenation of A_s , A , and a .

Similarly, register BR is subdivided into B_s , B , and b , and QR into Q_s , Q , and q . A parallel-adder adds the two mantissas and transfers the sum into A and the carry into E . A separate parallel-adder is used for the exponents. Since the exponents are biased, they do not have a distinct sign bit but are represented as a biased positive quantity. It is assumed that the floating-point numbers are so large that the chance of an exponent overflow is very remote, and for this reason the exponent overflow will be neglected. The exponents are also connected to a magnitude comparator that provides three binary outputs to indicate their relative magnitude.

The number in the mantissa will be taken as a *fraction*, so the binary point is assumed to reside to the left of the magnitude part. Integer representation for floating-point causes certain scaling problems during multiplication and division. To avoid these problems, we adopt a fraction representation.

The numbers in the registers are assumed to be initially normalized. After each arithmetic operation, the result will be normalized. Thus all floating-point operands coming from and going to the memory unit are always normalized.

Addition and Subtraction

During addition or subtraction, the two floating-point operands are in AC and BR . The sum or difference is formed in the AC . The algorithm can be divided into four consecutive parts:

1. Check for zeros.
2. Align the mantissas.
3. Add or subtract the mantissas.
4. Normalize the result.

A floating-point number that is zero cannot be normalized. If this number is used during the computation, the result may also be zero. Instead of checking for zeros during the normalization process we check for zeros at the beginning and terminate the process if necessary. The alignment of the mantissas must be carried out prior to their operation. After the mantissas are added or subtracted, the result may be unnormalized. The normalization procedure ensures that the result is normalized prior to its transfer to memory.

The flowchart for adding or subtracting two floating-point binary numbers is shown in Fig. 10-15. If BR is equal to zero, the operation is terminated, with the value in the AC being the result. If AC is equal to zero, we transfer

the content of BR into AC and also complement its sign if the numbers are to be subtracted. If neither number is equal to zero, we proceed to align the mantissas.

The magnitude comparator attached to exponents a and b provides three outputs that indicate their relative magnitude. If the two exponents are equal, we go to perform the arithmetic operation. If the exponents are not equal, the mantissa having the smaller exponent is shifted to the right and its exponent incremented. This process is repeated until the two exponents are equal.

The addition and subtraction of the two mantissas is identical to the fixed-point addition and subtraction algorithm presented in Fig. 10-2. The magnitude part is added or subtracted depending on the operation and the signs of the two mantissas. If an overflow occurs when the magnitudes are added, it is transferred into flip-flop E . If E is equal to 1, the bit is transferred into A_1 and all other bits of A are shifted right. The exponent must be incremented to maintain the correct number. No underflow may occur in this case because the original mantissa that was not shifted during the alignment was already in a normalized position.

If the magnitudes were subtracted, the result may be zero or may have an underflow. If the mantissa is zero, the entire floating-point number in the AC is made zero. Otherwise, the mantissa must have at least one bit that is equal to 1. The mantissa has an underflow if the most significant bit in position A_1 is 0. In that case, the mantissa is shifted left and the exponent decremented. The bit in A_1 is checked again and the process is repeated until it is equal to 1. When $A_1 = 1$, the mantissa is normalized and the operation is completed.

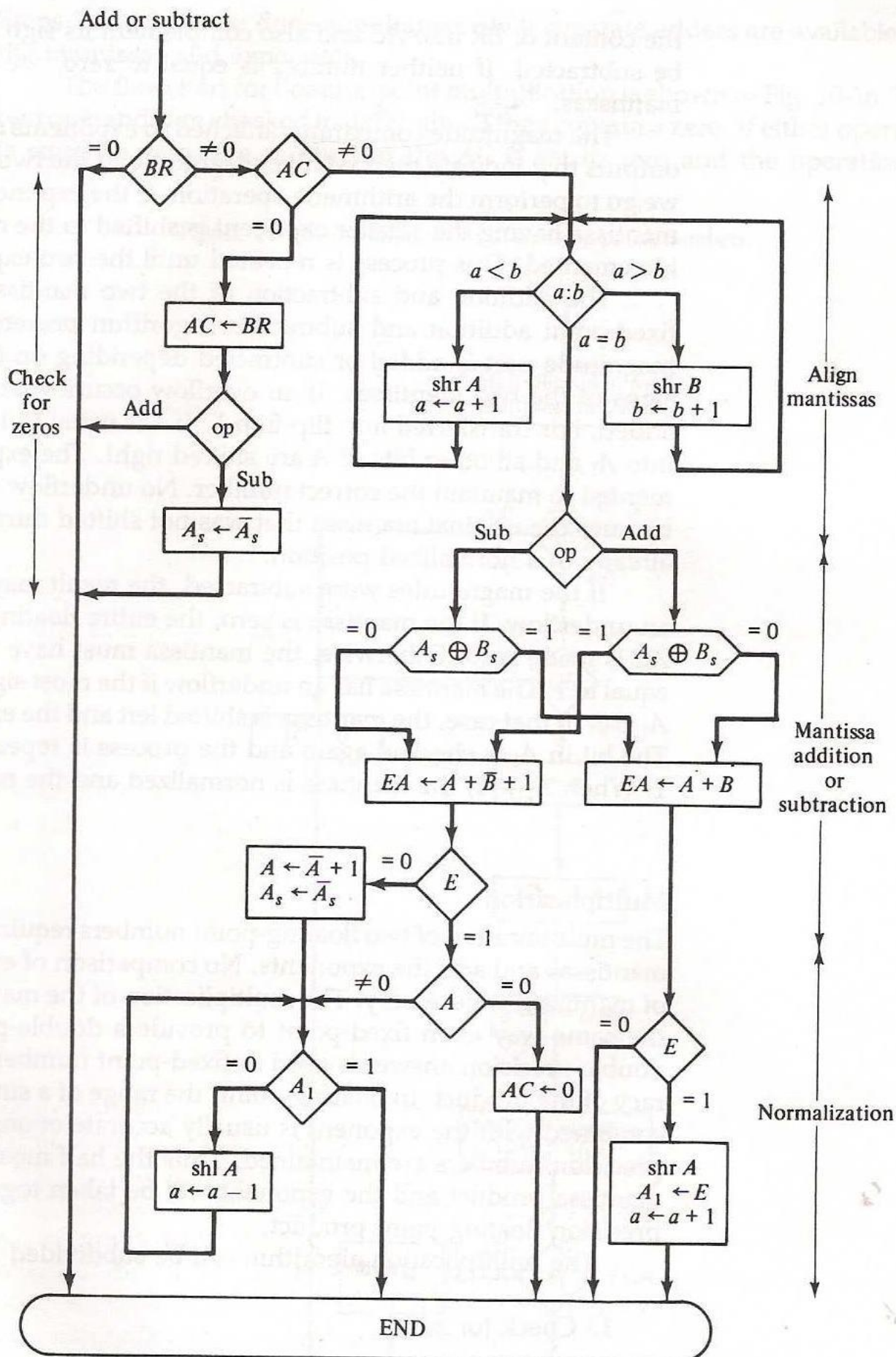


Figure 10-15 Addition and subtraction of floating-point numbers.